

Sistema Metro Mondego

Bruno Miguel Santos Marques – uc2018278019@student.uc.pt

David Ramos – uc2013143286@student.uc.pt

João Vítor Alves – uc2016122878@student.uc.pt

Coimbra, 2026

Índice

1. Breve Descrição do Projeto	3
1.1. Definição Detalhada de Transações, Conflitos de Concorrência e Estratégias de Mitigação.....	3
1.2. Plano de Desenvolvimento (Tarefas Planeadas, Divisão Inicial de Trabalho e Cronograma)	4
2. Diagrama Entidade-Relacionamento (ER)	5
2.1. Modelo Conceptual	5
2.2. Descrição de Entidades, Atributos e Regras de Integridade.....	6
3. Modelo de Dados Relacional (modelo físico)	8

1. Breve Descrição do Projeto

O presente projeto tem como objetivo conceber e implementar o sistema de gestão de base de dados para o "Metro Mondego", uma empresa inovadora de mobilidade urbana na região de Coimbra. O sistema baseia-se num motor relacional PostgreSQL, cujas funcionalidades serão expostas através de uma REST API em Python. A arquitetura de dados desenvolvida visa suportar operações diárias essenciais, tais como a gestão da rede física (linhas e estações), o processamento transacional de venda e validação de títulos de transporte, a gestão de carteiras virtuais dos clientes e o registo histórico de tarifários e promoções para efeitos analíticos. Todo o modelo foi desenhado de forma a garantir elevados níveis de integridade e controlo de concorrência, para garantir a fiabilidade das operações.

1.1. Definição Detalhada de Transações, Conflitos de Concorrência e Estratégias de Mitigação

Apesar de o DBMS (PostgreSQL) gerir automaticamente anomalias básicas através do seu nível de isolamento padrão (*Read Committed*), o sistema lida com regras de negócio que exigem um controlo de concorrência explícito, não acautelado nativamente pelo motor:

Transação A: Compra de Bilhete e Desconto de Saldo

- **Definição:** O processo implica a leitura do saldo atual (`wallet_balance`) do Cliente, validação de fundos face ao preço da viagem, atualização (`UPDATE`) com a dedução do saldo, e inserção (`INSERT`) dos novos registos nas tabelas `Ticket` e `Wallet_Transaction`.
- **Conflito de Concorrência (*Lost Update*):** Se um cliente submeter dois pedidos de compra simultâneos, duas transações paralelas farão a leitura do mesmo saldo inicial, passando na verificação de fundos. Ambas deduzirão o valor sobre o saldo original lido, permitindo a emissão de múltiplos bilhetes sem que a conta reflita a totalidade da despesa (fenómeno de *Lost Update*, decorrente de um *Non-repeatable Read* não mitigado).

- **Estratégia de Mitigação:** Será aplicado um bloqueio explícito pessimista através de SELECT ... FOR UPDATE aquando da leitura do saldo na tabela Customer. Isto cria um bloqueio ao nível da linha (*Row-level Lock*) que obriga as transações concorrentes a aguardar que a primeira conclua o COMMIT, forçando a segunda transação a ler o novo saldo já descontado.

Transação B: Validação e Controlo de Lotação (*Overbooking*)

- **Definição:** Para autorizar uma Validation, o sistema tem de ler e contar as validações existentes para aquela viagem num dado momento, garantindo que o número total é inferior ao limite estipulado em `vehicle_capacity`. Só após esta verificação o registo da passagem é inserido.
- **Conflito de Concorrência (*Write Skew*):** Tratando-se de um problema de leitura agregada, se restarem apenas 2 lugares vagos e 3 clientes picarem o bilhete em torniquetes diferentes exatamente no mesmo milissegundo, as 3 transações lerão a mesma ocupação simultaneamente. Como as leituras cruzadas aprovam a inserção dos 3 utilizadores, o veículo ultrapassará o limite de lotação (anomalia de *Write Skew* provocada por leituras fantasma / *Phantom Reads*).
- **Estratégia de Mitigação:** A transação será executada com o nível de isolamento elevado forçosamente para SERIALIZABLE. Este modo obriga as transações a comportarem-se de forma estritamente sequencial, levando o SGBD a abortar proativamente (com erro de serialização) qualquer transação concorrente que afete a base de cálculo de uma transação ainda em curso.

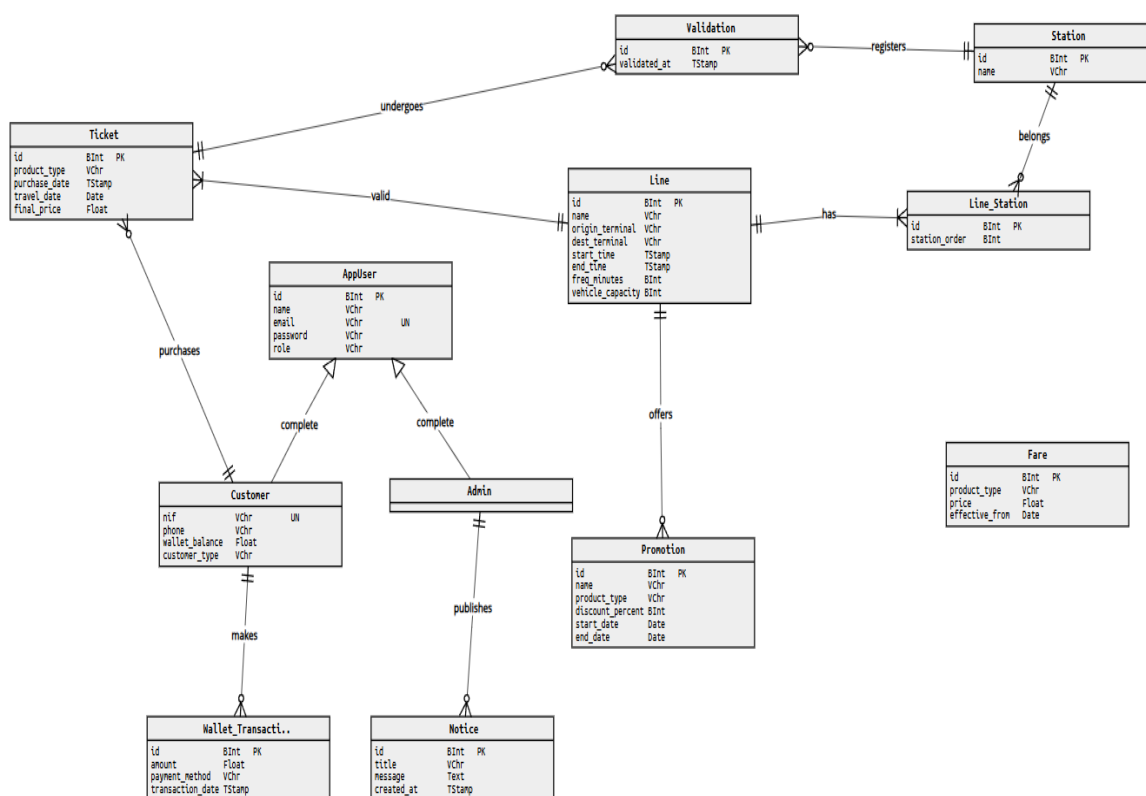
1.2. Plano de Desenvolvimento (Tarefas Planeadas, Divisão Inicial de Trabalho e Cronograma)

Inicialmente, a criação e discussão dos modelos relacionais e físicos foram um esforço conjunto feito em horários fora de aula, esforço esse maior presente do participante Bruno. A descrição

A divisão de trabalho ficou mais complexa na próxima fase, sendo dividida de forma a garantir a maior celeridade do projeto. Timeline: com o percurso do tempo, a maior parte do projeto foi criada.

Na última semana, os integrantes João e Bruno fizeram aproximadamente 8h e 5h de trabalho, respectivamente, sozinhos, e cerca de 1-2h de trabalho conjunto com todos os participantes

2. Diagrama Entidade-Relacionamento (ER)



O modelo conceitual foi desenvolvido com recurso à plataforma ONDA, refletindo os requisitos operacionais do Metro Mondego.

2.1. Modelo Conceptual

2.2. Descrição de Entidades, Atributos e Regras de Integridade

Decisões Arquiteturais de Modelação:

- **Ausência de Entidades Fracas (Promoção a Entidades Fortes):** De um ponto de vista puramente conceptual, entidades como a *Validation*, a *Wallet_Transaction* e a *Line_Station* seriam classificadas como Entidades Fracas (pois a sua existência depende logicamente do *Ticket*, do *Customer* e da *Line/Station*, respetivamente). No entanto, optou-se por desenhar todo o diagrama com Entidades Fortes através da introdução de Chaves Substitutas artificiais (o atributo numérico *id* em todas as tabelas). Esta decisão evita a propagação de chaves compostas pesadas no modelo físico relacional, otimiza as queries analíticas e facilita o roteamento nos *endpoints* da futura *REST API*.
- **Modelação do "Super Administrador":** O enunciado prevê a existência de um *Super Administrator* como uma conta única *hard-coded*. Evitou-se propositadamente a criação de uma entidade filha isolada para este papel, pois resultaria numa tabela fisicamente ineficiente (apenas com um *ID*, sem atributos extra). Esta diferenciação é antes tratada ao nível lógico na entidade pai.

Entidades e Atributos Principais:

- **AppUser:** Entidade que gere a autenticação central (email único, password codificada). O atributo *role* é um enumerador ('SuperAdmin', 'Admin', 'Customer'), acomodando os requisitos de permissões e acessos do sistema de forma consolidada.
- **Customer e Admin:** Entidades filhas que herdam de AppUser. O Cliente aloja dados financeiros e fiscais sensíveis (*nif*, *wallet_balance*) e ainda define quem é o cliente para fins tarifários (presentes no atributo *customer_type*). O Administrador garante a auditoria de autoria em operações logísticas.

- **Line e Station:** A infraestrutura. A Linha detém os parâmetros de operação dinâmicos (ex: `vehicle_capacity`, `freq_minutes`). A entidade associativa **Line_Station** resolve a relação N:M entre ambas e aloja o atributo `station_order`, garantindo que cada linha possui uma lista ordenada de paragens.
- **Fare e Promotion:** O modelo económico. A tabela de tarifário (Fare) preserva o histórico previsional com inserções baseadas no atributo `effective_from`. Propositadamente, não possui chaves estrangeiras para bilhetes antigos, garantindo o isolamento histórico dos dados.
- **Ticket:** O título de transporte adquirido, guardando de forma permanente o `final_price` efetivamente transacionado, imune a alterações futuras de preço.
- **Validation:** Regista a utilização de um bilhete (picar no torniquete), cruzando temporalmente a validação (`validated_at`) com o local físico (Station).
- **Wallet_Transaction:** Histórico imutável de movimentos (carregamentos) efetuados na carteira do cliente.
- **Notice:** Avisos e comunicados globais redigidos pelos administradores.

Regras de Integridade:

- **Herança Completa (*Complete/Mandatory*):** A criação de um Customer ou Admin exige sempre a pré-existência mandatória de um registo de credenciais válidas na entidade pai AppUser.
- **Integridade Referencial Estrita (*Foreign Keys*):** Aplicada de forma global. Por exemplo, um Ticket apenas pode ser emitido no sistema se referenciar um Customer validado e uma Line ativa.
- **Restrições de Domínio (*Check Constraints*):** Atributos de natureza financeira ou quantitativa terão a sua coerência garantida ao nível físico (ex: `wallet_balance >= 0`, `final_price >= 0`, `amount > 0`), impossibilitando valores negativos na base de dados.

3. Modelo de Datos Relacional (modelo físico)

